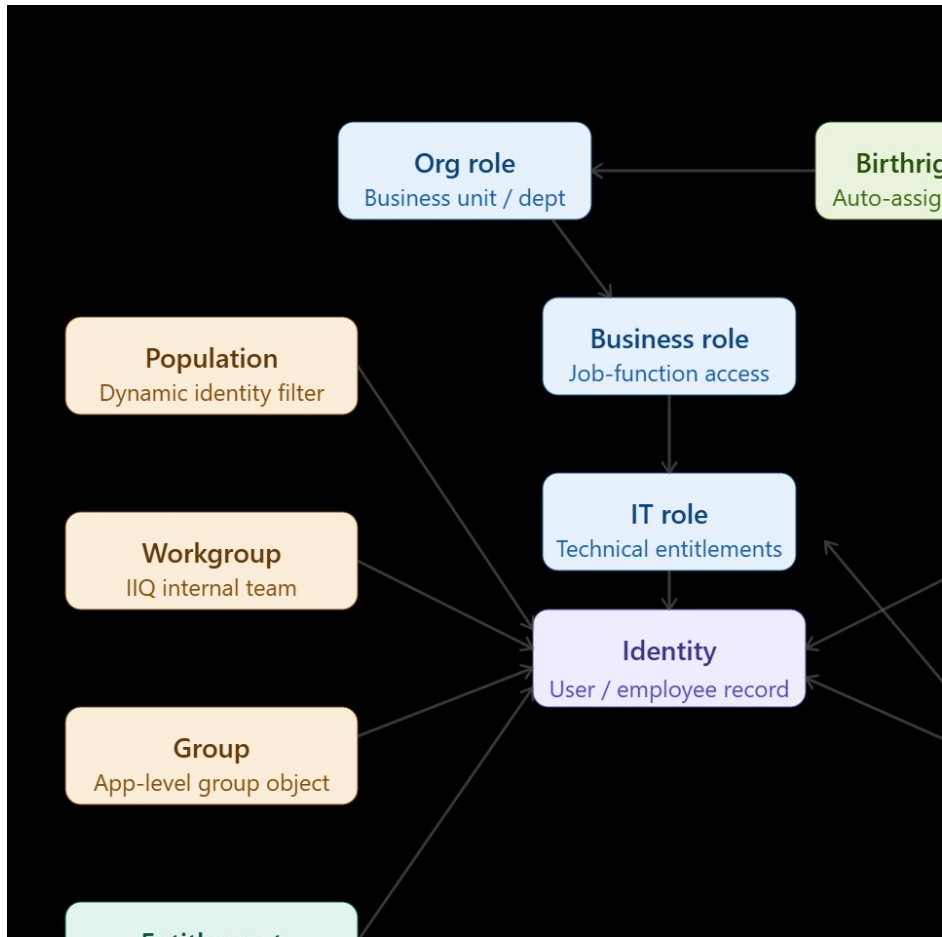


Sailpoint definitions

Let me build a comprehensive reference diagram to accompany the definitions. Here is a precise breakdown of each term, grounded in IIQ 8.5 concepts, with banking examples.



Entitlement

An entitlement is a specific access value on a managed application — think of it as a single row in an account attribute. In IIQ, these are harvested during aggregation and stored as `ManagedAttribute` objects.

Analogy: If an application is a database table, an entitlement is one cell value in the "access level" column.

Banking example: On the core banking system, the group membership `ROLE_TELLER_CASH_LIMIT_5K` is an entitlement. It means "this account has that group assigned."

Permission

A permission is a sub-object within an entitlement, representing a **target + rights** pair — similar to a Unix ACL entry. It is most common in connectors that expose object-level permissions (e.g. Active Directory, Unix).

Analogy: If the entitlement is "access to the vault room," the permission is "vault room door → read, open, lock."

Banking example: On a Unix file server hosting SWIFT message archives, a permission might be `target=/swift/outgoing, rights=read,write`.

Capability

A capability controls what a user can **do inside the IIQ UI itself** — it is not about target application access. It is essentially a named bundle of IIQ UI feature flags (view certifications, run reports, manage roles, etc.).

Analogy: In a Java Spring app, capabilities are like method-level `@PreAuthorize` annotations applied to the IIQ admin console pages.

Banking example: The identity governance team lead gets the `Full Access Administrator` capability in IIQ, while an auditor gets `View-only Certifier` — both are bank employees, but their IIQ UI surfaces are different.

Group

A group is an account-level grouping object that exists on a **target application** (e.g. an AD group, an Oracle DB role, an LDAP `groupOfNames`). IIQ aggregates these as `ManagedAttribute` objects of type `group`.

Banking example: The Active Directory group `GG_SWIFT_OPERATORS` is a group in IIQ. Identities whose accounts are members of it carry that group as an entitlement.

Workgroup

A workgroup is an **IIQ-internal team construct** (a special `Identity` of type `workgroup`). It is used to assign ownership of certification campaigns, roles, or policies — not to grant access to target systems. It has no counterpart in connected applications.

Analogy: A workgroup is like a distribution list in your internal ticketing system — it exists only inside the tool, not in the applications the tool manages.

Banking example: The workgroup `WG_IT_Risk_Owners` owns all high-risk access certifications for the trading floor. If a role needs approval, IIQ routes the work item to this workgroup.

Population

A population is a **saved identity search / dynamic filter** — it yields a snapshot set of identities matching criteria at a point in time. Populations are used in certifications, reports, and provisioning policies to scope which identities are targeted.

Analogy: A population is like a SQL `VIEW` over the `spt_identity` table — it is a reusable query, not a stored set of rows.

Banking example: The population `POP_Branch_Tellers_EMEA` might filter on `department = "Retail Banking" AND region = "EMEA"`. When a certification campaign runs, it targets exactly these identities.

Profile

A profile is a filter attached to an **IT role** that defines which entitlements satisfy the role. When IIQ evaluates whether an identity "has" a role, it checks whether the identity's account attributes match the profile criteria.

Analogy: A profile is like a `WHERE` clause in a `JOIN` between the identity's accounts and the entitlement table — the role is granted if the row matches.

Banking example: The IT role `ITR_Core_Banking_ReadOnly` has a profile: `application = "Finacle" AND entitlement = "ROLE_INQUIRY_ONLY"`. Any account matching that filter is considered to have the role.

Business Role

A business role represents a **job function** in business language, independent of any specific application. It is composed of one or more IT roles (and optionally other business roles). Business roles are what end users and managers understand.

Banking example: `BR_Retail_Teller` is a business role. It contains IT roles for the core banking system, the document scanning system, and the branch intranet. A teller gets all three when assigned this role.

IT Role

An IT role is a **technical role** that groups entitlements on one or more target applications via profiles. IT roles are the bridge between business language (business roles) and raw entitlements on applications.

Banking example: `ITR_Finacle_Teller_Access` is an IT role. It has a profile requiring `group = GG_FINACLE_TELLERS` on the Finacle connector. It is assigned to users, or included in a business role.

Organizational Role

An organizational role (Org role) models **company structure** — department, division, cost center, geography. It sits above business roles in the role hierarchy and is typically used to auto-assign business roles based on HR attributes.

Banking example: `ORG_Retail_Banking_DACH` is an org role. Any identity with `division = "Retail Banking" AND country IN (DE, AT, CH)` gets this role, which in turn includes the `BR_Retail_Teller` business role.

Permitted Role

A permitted role is a role declared **inside another role's definition** as optionally requestable. If role A lists role B as "permitted," then any identity holding role A is *allowed to request* role B — but does not receive it automatically.

Analogy: Think of permitted roles as optional modules in a university curriculum. If you are enrolled in the Finance track (role A), you are permitted to take the Advanced Derivatives elective (role B), but it is not mandatory.

Banking example: The business role `BR_Retail_Teller` has `BR_Safe_Deposit_Access` as a permitted role. Tellers do not automatically get safe-deposit access, but they can request it through the IIQ access request portal.

Allowed Role

An allowed role is declared in a role definition to specify which roles an identity holding that role **is eligible to also hold** without triggering a Separation of Duties (SoD) violation. It is a compatibility/whitelist declaration.

Analogy: Allowed roles are like pre-approved combinations in a conflict-of-interest policy — "it is acceptable for this person to hold both role A and role B simultaneously."

Banking example: The policy might normally flag someone having both `BR_Retail_Teller` and `BR_Loan_Officer` as a conflict. But if `BR_Loan_Officer` is declared as an allowed role within a special `BR_Universal_Banker` role, identities in that role can hold both without triggering a violation.

Birthright Role / `birthRight` (attribute)

These are two related but distinct things:

Birthright role is a role automatically assigned to an identity upon creation or based on HR join event — no request or approval needed. It defines the baseline access every employee of a given type should have on day one.

`birthRight` (lowercase, as an attribute on a role assignment) is a boolean flag on the `RoleAssignment` or `Bundle` object in IIQ's data model. When `true`, it marks that a particular role assignment was granted automatically by provisioning policy, not by manual request. This flag protects the assignment from being removed in certifications unless the underlying policy criteria change.

Analogy: The birthright role is the employment contract clause "you get a laptop and VPN access on day one." The `birthRight` flag is the stamp on the provisioning record saying "this was auto-issued, do not revoke manually."

Banking example: Every new hire in the bank gets BR_Corporate_Baseline (email, intranet, HR self-service) as a birthright role. In the database, the `spt_identity_role` record for this assignment has `birthright = 1`. When the annual recertification runs, a certifier cannot accidentally revoke it unless the HR feed no longer qualifies the user.

Quick summary table

Term	Scope	Lives in	Assigned to
Entitlement	Target app	ManagedAttribute	Account
Permission	Target app (ACL)	Within entitlement	Account
Group	Target app	ManagedAttribute (group)	Account
Capability	IIQ UI	IIQ config	Identity (admin)
Workgroup	IIQ internal	Identity (type=workgroup)	Work items / ownership
Population	IIQ query	Saved search	Used in campaigns/reports
Profile	Role definition	IT Role	Criteria for role detection
IT Role	Technical	Bundle	Identity / Business Role
Business Role	Business	Bundle	Identity / Org Role
Org Role	Structural	Bundle	Identity (via HR attributes)
Permitted Role	Role catalog	Inside another role	Requestable by role holder
Allowed Role	SoD policy	Inside another role	Concurrent eligibility
Birthright Role	Provisioning	Bundle + policy	Identity automatically

Let's say I will modify the table structure to add validity dates in order to track in a separate table attributes change for a ROBID, like an SCD type 2. What should be modified

1/ in the DDL create

2/ Sql loader

3/ will the use case make use of MERGE statement

Q: Where do you want to store the SCD2 history? A: Both — current in STG, history in HIST table

Q: What triggers a new historical record? A: Specific columns only (e.g. ENTITY, MANAGER, FUNCTION)

Q: How is the flat file delivered — full snapshot each time or delta only? A: Full snapshot every load

The analogy

Think of it like a **passport system**:

- STG_MASTER_ROBI = the **current valid passport** (one row per person)
- STG_MASTER_ROBI_HIST = the **passport archive** (all old versions with their validity period)
- SQL*Loader = the **border scanner** that loads the new snapshot
- MERGE = the **clerk** who compares old vs new and decides what to stamp

1/ DDL Changes

Modify STG_MASTER_ROBI — add SCD2 tracking columns:

```
ALTER TABLE OWN_AP12008.STG_MASTER_ROBI ADD (  
  
    VALID_FROM DATE DEFAULT SYSDATE NOT NULL,  
  
    VALID_TO DATE DEFAULT DATE '9999-12-31' NOT NULL,  
  
    IS_CURRENT VARCHAR2(1) DEFAULT 'Y' NOT NULL  
  
);
```

Create STG_MASTER_ROBI_HIST — identical structure:

sql

```
CREATE TABLE OWN_AP12008.STG_MASTER_ROBI_HIST AS  
  
SELECT * FROM OWN_AP12008.STG_MASTER_ROBI
```

WHERE 1=0;

2/ SQL*Loader changes

SQL*Loader's role **stays simple** — it loads the full snapshot into a **staging landing table** first, not directly into STG_MASTER_ROBI anymore.

-- New landing/staging table, no SCD columns needed

```
CREATE TABLE OWN_AP12008.STG_MASTER_ROBI_LANDING AS

SELECT ROBIID, FLID, START_DATE, END_DATE, ENTITYID, ENTITY,

        MANAGER, LASTNAME, FIRSTNAME, COUNTRY, LANGUAGE,

        FUNCTION, REFOG, EMAIL, TYPE, STATUS, ITEST, ISRPA

FROM OWN_AP12008.STG_MASTER_ROBI

WHERE 1=0;
```

Control file — point to the landing table:

```
LOAD DATA

CHARACTERSET UTF8

TRUNCATE

INTO TABLE OWN_AP12008.STG_MASTER_ROBI_LANDING

FIELDS TERMINATED BY '|'

OPTIONALLY ENCLOSED BY '"'

TRAILING NULLCOLS

(

    ROBIID,

    FLID,

    START_DATE,

    END_DATE,

    ENTITYID,
```



```
ENTITY,  
  
MANAGER,  
  
LASTNAME,  
  
FIRSTNAME,  
  
COUNTRY,  
  
LANGUAGE,  
  
FUNCTION,  
  
REFOG,  
  
EMAIL,  
  
TYPE,  
  
STATUS,  
  
ISTEST,  
  
ISRPA  
)
```

No SOURCE, CREATED_ON, VALID_FROM etc. here — those are handled by the **MERGE**.

3/ MERGE statement — yes, absolutely

This runs **after** SQL*Loader completes. Two steps:

Step 1 — Close expired records into HIST:

-- Archive rows where tracked columns have changed

```
INSERT INTO OWN_AP12008.STG_MASTER_ROBI_HIST
```

```
SELECT h.*
```

```
FROM OWN_AP12008.STG_MASTER_ROBI h
```

```

JOIN OWN_AP12008.STG_MASTER_ROBI_LANDING I
    ON h.ROBID = I.ROBID
WHERE h.IS_CURRENT = 'Y'
AND (
    h.ENTITY != I.ENTITY OR
    h.MANAGER != I.MANAGER OR
    h.FUNCTION != I.FUNCTION
    -- add more tracked columns here
);

-- Expire those same rows in the current table
UPDATE OWN_AP12008.STG_MASTER_ROBI h
SET h.IS_CURRENT = 'N',
    h.VALID_TO = TRUNC(SYSDATE)
WHERE h.IS_CURRENT = 'Y'
AND EXISTS (
    SELECT 1
    FROM OWN_AP12008.STG_MASTER_ROBI_LANDING I
    WHERE I.ROBID = h.ROBID
    AND (
        h.ENTITY != I.ENTITY OR
        h.MANAGER != I.MANAGER OR
        h.FUNCTION != I.FUNCTION
    )
);

```

Step 2 — MERGE new/changed rows into STG:

```
MERGE INTO OWN_AP12008.STG_MASTER_ROBI tgt
USING (
    SELECT I.*
    FROM OWN_AP12008.STG_MASTER_ROBI_LANDING I
) src
ON (tgt.ROBID = src.ROBID AND tgt.IS_CURRENT = 'Y')

WHEN MATCHED THEN
    -- Update non-tracked columns only (tracked ones got a new row)
    UPDATE SET
        tgt.FLID    = src.FLID,
        tgt.EMAIL   = src.EMAIL,
        tgt.STATUS  = src.STATUS,
        tgt.UPDATED_ON = SYSDATE
    WHERE (
        tgt.ENTITY = src.ENTITY AND
        tgt.MANAGER = src.MANAGER AND
        tgt.FUNCTION = src.FUNCTION
        -- i.e. only update if NO tracked change (tracked changes = new row below)
    )

WHEN NOT MATCHED THEN
```

-- New ROBID or new version after expiry

```
INSERT (  
    ROBID, FLID, START_DATE, END_DATE, ENTITYID, ENTITY,  
    MANAGER, LASTNAME, FIRSTNAME, COUNTRY, LANGUAGE,  
    FUNCTION, REFOG, EMAIL, TYPE, STATUS, ITEST, ISRPA,  
    SOURCE, CREATED_ON, UPDATED_ON, VALID_FROM, VALID_TO, IS_CURRENT  
)  
VALUES (  
    src.ROBID, src.FLID, src.START_DATE, src.END_DATE,  
    src.ENTITYID, src.ENTITY, src.MANAGER, src.LASTNAME,  
    src.FIRSTNAME, src.COUNTRY, src.LANGUAGE, src.FUNCTION,  
    src.REFOG, src.EMAIL, src.TYPE, src.STATUS,  
    src.ITEST, src.ISRPA,  
    'ROBI', SYSDATE, SYSDATE, TRUNC(SYSDATE), DATE '9999-12-31', 'Y'  
);
```

Execution order summary

1. sqlldr → truncate & load LANDING table
2. Step 1 → archive changed rows to HIST, expire in STG
3. Step 2 → MERGE LANDING into STG
4. COMMIT

Here is the complete stored procedure:

```
CREATE OR REPLACE PROCEDURE OWN_AP12008.SP_LOAD_STG_MASTER_ROBI  
AS
```

-- Counters for logging

```
v_archived  NUMBER := 0;

v_expired   NUMBER := 0;

v_merged    NUMBER := 0;

v_proc_name VARCHAR2(50) := 'SP_LOAD_STG_MASTER_ROBI';

v_start_time TIMESTAMP := SYSTIMESTAMP;
```

BEGIN

```
DBMS_OUTPUT.PUT_LINE(v_proc_name || ' started at: ' ||
    TO_CHAR(v_start_time, 'YYYY-MM-DD HH24:MI:SS'));
```

```
-- =====
```

```
-- STEP 1A: Archive changed rows into HIST
```

```
-- Think of it as: photocopying the old passport page
```

```
-- before stamping a new one
```

```
-- =====
```

```
INSERT INTO OWN_AP12008.STG_MASTER_ROBI_HIST
```

```
SELECT h.*
```

```
FROM  OWN_AP12008.STG_MASTER_ROBI h
```

```
JOIN  OWN_AP12008.STG_MASTER_ROBI_LANDING l
```

```
      ON h.ROBID = l.ROBID
```

```
WHERE h.IS_CURRENT = 'Y'
```

```
AND (
```

```
      NVL(h.ENTITY, '~NULL~') != NVL(l.ENTITY, '~NULL~') OR
```

```
      NVL(h.MANAGER, '~NULL~') != NVL(h.MANAGER, '~NULL~') OR
```

```

        NVL(h.FUNCTION, '~NULL~') != NVL(l.FUNCTION, '~NULL~')

);

v_archived := SQL%ROWCOUNT;

DBMS_OUTPUT.PUT_LINE('Step 1A - Rows archived to HIST: ' || v_archived);

-- =====

-- STEP 1B: Expire those same rows in STG

-- Think of it as: stamping EXPIRED on the old passport

-- =====

UPDATE OWN_AP12008.STG_MASTER_ROBI h

SET   h.IS_CURRENT = 'N',

      h.VALID_TO   = TRUNC(SYSDATE),

      h.UPDATED_ON = SYSDATE

WHERE h.IS_CURRENT = 'Y'

AND EXISTS (

    SELECT 1

    FROM OWN_AP12008.STG_MASTER_ROBI_LANDING l

    WHERE l.ROBID = h.ROBID

    AND (

        NVL(h.ENTITY, '~NULL~') != NVL(l.ENTITY, '~NULL~') OR

        NVL(h.MANAGER, '~NULL~') != NVL(l.MANAGER, '~NULL~') OR

        NVL(h.FUNCTION, '~NULL~') != NVL(l.FUNCTION, '~NULL~')

    )

);

```

```

v_expired := SQL%ROWCOUNT;

DBMS_OUTPUT.PUT_LINE('Step 1B - Rows expired in STG: ' || v_expired);

-- =====

-- STEP 1C: Archive and expire rows for ROBIDs
-- that disappeared from the new snapshot entirely
-- Think of it as: passport holder left the company
-- =====

INSERT INTO OWN_AP12008.STG_MASTER_ROBI_HIST
SELECT h.*
FROM OWN_AP12008.STG_MASTER_ROBI h
WHERE h.IS_CURRENT = 'Y'
AND NOT EXISTS (
    SELECT 1
    FROM OWN_AP12008.STG_MASTER_ROBI_LANDING I
    WHERE I.ROBID = h.ROBID
);

DBMS_OUTPUT.PUT_LINE('Step 1C - Departed ROBIDs archived: ' || SQL%ROWCOUNT);

UPDATE OWN_AP12008.STG_MASTER_ROBI h
SET h.IS_CURRENT = 'N',
    h.VALID_TO = TRUNC(SYSDATE),
    h.UPDATED_ON = SYSDATE

```

```
WHERE h.IS_CURRENT = 'Y'
```

```
AND NOT EXISTS (
```

```
    SELECT 1
```

```
    FROM OWN_AP12008.STG_MASTER_ROBI_LANDING I
```

```
    WHERE I.ROBID = h.ROBID
```

```
);
```

```
DBMS_OUTPUT.PUT_LINE('Step 1C - Departed ROBIDs expired: ' || SQL%ROWCOUNT);
```

```
-- =====
```

```
-- STEP 2: MERGE landing into STG
```

```
-- WHEN MATCHED + no tracked change = update minor fields
```

```
-- WHEN NOT MATCHED = brand new ROBID or new version
```

```
-- =====
```

```
MERGE INTO OWN_AP12008.STG_MASTER_ROBI tgt
```

```
USING (
```

```
    SELECT * FROM OWN_AP12008.STG_MASTER_ROBI_LANDING
```

```
) src
```

```
ON (tgt.ROBID = src.ROBID AND tgt.IS_CURRENT = 'Y')
```

```
WHEN MATCHED THEN
```

```
    UPDATE SET
```

```
        tgt.FLID    = src.FLID,
```

```
        tgt.START_DATE = src.START_DATE,
```

```
        tgt.END_DATE  = src.END_DATE,
```



```

tgt.ENTITYID = src.ENTITYID,

tgt.LASTNAME = src.LASTNAME,

tgt.FIRSTNAME = src.FIRSTNAME,

tgt.COUNTRY = src.COUNTRY,

tgt.LANGUAGE = src.LANGUAGE,

tgt.REFOG = src.REFOG,

tgt.EMAIL = src.EMAIL,

tgt.TYPE = src.TYPE,

tgt.STATUS = src.STATUS,

tgt.ISTEST = src.ISTEST,

tgt.ISRPA = src.ISRPA,

tgt.UPDATED_ON = SYSDATE

-- Only update if NO tracked column changed

-- (tracked changes already got IS_CURRENT='N' in step 1B)

WHERE (

    NVL(tgt.ENTITY, '~NULL~') = NVL(src.ENTITY, '~NULL~') AND

    NVL(tgt.MANAGER, '~NULL~') = NVL(src.MANAGER, '~NULL~') AND

    NVL(tgt.FUNCTION, '~NULL~') = NVL(src.FUNCTION, '~NULL~')

)

WHEN NOT MATCHED THEN

INSERT (

    ROBID, FLID, START_DATE, END_DATE, ENTITYID,

    ENTITY, MANAGER, LASTNAME, FIRSTNAME, COUNTRY,

    LANGUAGE, FUNCTION, REFOG, EMAIL, TYPE,

```

```

        STATUS, ITEST, ISRPA,

        SOURCE, CREATED_ON, UPDATED_ON,

        VALID_FROM, VALID_TO, IS_CURRENT
    )

    VALUES (

        src.ROBID, src.FLID, src.START_DATE, src.END_DATE, src.ENTITYID,

        src.ENTITY, src.MANAGER, src.LASTNAME, src.FIRSTNAME, src.COUNTRY,

        src.LANGUAGE, src.FUNCTION, src.REFOG, src.EMAIL, src.TYPE,

        src.STATUS, src.ITEST, src.ISRPA,

        'ROBI', SYSDATE, SYSDATE,

        TRUNC(SYSDATE), DATE '9999-12-31', 'Y'

    );

```

```

v_merged := SQL%ROWCOUNT;

```

```

DBMS_OUTPUT.PUT_LINE('Step 2 - Rows merged into STG: ' || v_merged);

```

```

-- =====

```

```

-- COMMIT everything atomically

```

```

-- All steps succeed or none do

```

```

-- =====

```

```

COMMIT;

```

```

DBMS_OUTPUT.PUT_LINE(v_proc_name || ' completed successfully at: ' ||

    TO_CHAR(SYSTIMESTAMP, 'YYYY-MM-DD HH24:MI:SS'));

```

EXCEPTION

WHEN OTHERS THEN

ROLLBACK;

DBMS_OUTPUT.PUT_LINE(v_proc_name || ' FAILED: ' || SQLERRM);

-- Re-raise so the calling shell script gets a non-zero exit

RAISE;

END SP_LOAD_STG_MASTER_ROBI;

/

Updated shell script — wire it all together

#!/bin/bash

set -e # exit on any error

DB_PASS=\$(vault kv get -field=password secret/oracle/own_ap12008)

CONNECT="OWN_AP12008/\${DB_PASS}@d00163c6-m1.dbs.xod.net.intra:1521/D00163C6_SERVICE"

echo "Step 1: Loading flat file into LANDING table..."

sqlldr userid="\${CONNECT}" parfile=load_stg_master_robi.par

if [\$? -ne 0]; then

echo "ERROR: SQL*Loader failed. Aborting."

exit 1

fi

```

echo "Step 2: Running SCD2 stored procedure..."

sqlplus -s "${CONNECT}" <<EOF

SET SERVEROUTPUT ON SIZE UNLIMITED

EXEC OWN_AP12008.SP_LOAD_STG_MASTER_ROBI;

EXIT SQL.SQLCODE;

EOF

```

echo "Done."

Key design notes

Decision	Reason
<code>NVL (col, '~NULL~')</code>	Handles NULL vs NULL comparison safely — plain <code>!=</code> returns NULL when either side is NULL
<code>ROLLBACK</code> on exception	All 3 steps are atomic — partial loads never corrupt the SCD2 chain
Step 1C (departed ROBIIDs)	Full snapshot means absence = departure — important to expire those too
<code>EXIT SQL.SQLCODE</code>	Shell script gets non-zero exit code if procedure fails

Is it possible to have a metadata/config table to flag which columns should be tracked ?

Excellent design thinking — this is a classic **configuration-driven** pattern, much more maintainable than hardcoding column names.

Before I write it, one quick clarification:

Q: The config table — how specific should it be? A: Per table + per source (in case other sources join later)

Q: How should the procedure read the tracked columns? A: Dynamic SQL (EXECUTE IMMEDIATE) — fully flexible but more complex

Perfect. Here is the full implementation.

Seed it for ROBI:

INSERT ALL

INTO OWN_AP12008.CFG_SCD2_TRACKED_COLUMNS

(TABLE_NAME, SOURCE_NAME, COLUMN_NAME, COMMENTS)

VALUES ('STG_MASTER_ROBI', 'ROBI', 'ENTITY', 'Business entity of person')

INTO OWN_AP12008.CFG_SCD2_TRACKED_COLUMNS

(TABLE_NAME, SOURCE_NAME, COLUMN_NAME, COMMENTS)

VALUES ('STG_MASTER_ROBI', 'ROBI', 'MANAGER', 'Manager REFOG code')

INTO OWN_AP12008.CFG_SCD2_TRACKED_COLUMNS

(TABLE_NAME, SOURCE_NAME, COLUMN_NAME, COMMENTS)

VALUES ('STG_MASTER_ROBI', 'ROBI', 'FUNCTION', 'Organisational function')

INTO OWN_AP12008.CFG_SCD2_TRACKED_COLUMNS

(TABLE_NAME, SOURCE_NAME, COLUMN_NAME, COMMENTS)

VALUES ('STG_MASTER_ROBI', 'ROBI', 'ENTITYID', 'Entity identifier')

SELECT 1 FROM DUAL;

COMMIT;

->Adding a new tracked column later = one INSERT into this table, zero code changes.

2/ Helper function — builds the WHERE clause dynamically

Think of this like a **clause factory** — the procedure calls it to get the comparison string freshly built from config each run.

CREATE OR REPLACE FUNCTION OWN_AP12008.FN_BUILD_SCD2_WHERE_CLAUSE (

p_table_name IN VARCHAR2,

p_source_name IN VARCHAR2,

```

    p_tgt_alias IN VARCHAR2, -- e.g. 'h' or 'tgt'

    p_src_alias IN VARCHAR2 -- e.g. 'l' or 'src'

)

RETURN VARCHAR2

AS

    v_clause VARCHAR2(4000) := '';

    v_col    VARCHAR2(100);

CURSOR c_cols IS

    SELECT COLUMN_NAME

    FROM   OWN_AP12008.CFG_SCD2_TRACKED_COLUMNS

    WHERE  TABLE_NAME = p_table_name

    AND    SOURCE_NAME = p_source_name

    AND    IS_ACTIVE   = 'Y'

    ORDER BY ID;

BEGIN

    FOR r IN c_cols LOOP

        IF v_clause IS NOT NULL THEN

            v_clause := v_clause || ' OR ';

        END IF;

        -- NVL trick handles NULL vs NULL safely

        -- like comparing two empty boxes:

        -- without NVL, NULL != NULL returns NULL not TRUE

        v_clause := v_clause ||

```

```

        'NVL(' || p_tgt_alias || ':' || r.COLUMN_NAME || ', "~NULL~") != ' ||
        'NVL(' || p_src_alias || ':' || r.COLUMN_NAME || ', "~NULL~")';

END LOOP;

IF v_clause IS NULL THEN

    RAISE_APPLICATION_ERROR(-20001,
        'No active tracked columns found for ' ||
        p_table_name || '/' || p_source_name);

END IF;

RETURN '(' || v_clause || ');

END FN_BUILD_SCD2_WHERE_CLAUSE;

/

3/ Updated stored procedure — fully dynamic

CREATE OR REPLACE PROCEDURE OWN_AP12008.SP_LOAD_STG_MASTER_ROBI
AS
    v_proc_name  VARCHAR2(50)  := 'SP_LOAD_STG_MASTER_ROBI';
    v_table_name  VARCHAR2(50)  := 'STG_MASTER_ROBI';
    v_source_name VARCHAR2(50)  := 'ROBI';

    -- Dynamic SQL blocks
    v_where_clause VARCHAR2(4000);
    v_sql          VARCHAR2(8000);

    -- Counters
    v_archived  NUMBER := 0;
    v_expired   NUMBER := 0;
    v_departed  NUMBER := 0;
    v_merged    NUMBER := 0;

```

BEGIN

```
DBMS_OUTPUT.PUT_LINE(v_proc_name || ' started at: ' ||  
    TO_CHAR(SYSTIMESTAMP, 'YYYY-MM-DD HH24:MI:SS'));
```

```
-- =====
```

```
-- Build the change detection WHERE clause from config
```

```
-- Result example:
```

```
-- (NVL(h.ENTITY, '~NULL~') != NVL(l.ENTITY, '~NULL~') OR
```

```
-- NVL(h.MANAGER, '~NULL~') != NVL(l.MANAGER, '~NULL~') ...)
```

```
-- =====
```

```
v_where_clause := OWN_AP12008.FN_BUILD_SCD2_WHERE_CLAUSE(  
    v_table_name, v_source_name, 'h', 'l');
```

```
DBMS_OUTPUT.PUT_LINE('Tracking WHERE clause: ' || v_where_clause);
```

```
-- =====
```

```
-- STEP 1A: Archive changed current rows to HIST
```

```
-- =====
```

```
v_sql :=
```

```
    'INSERT INTO OWN_AP12008.STG_MASTER_ROBI_HIST ' ||
```

```
    'SELECT h.* ' ||
```

```
    'FROM OWN_AP12008.STG_MASTER_ROBI h ' ||
```

```
    'JOIN OWN_AP12008.STG_MASTER_ROBI_LANDING l ' ||
```

```
    '    ON h.ROBID = l.ROBID ' ||
```

```
    'WHERE h.IS_CURRENT = "Y" ' ||
```

```
    'AND ' || v_where_clause;
```

```
EXECUTE IMMEDIATE v_sql;
```

```
v_archived := SQL%ROWCOUNT;
```

```
DBMS_OUTPUT.PUT_LINE('Step 1A - Archived to HIST: ' || v_archived);
```

```
-- =====
```

```
-- STEP 1B: Expire changed rows in STG
```

```
-- =====
```

```
v_sql :=
```

```
    'UPDATE OWN_AP12008.STG_MASTER_ROBI h ' ||
```

```
    'SET h.IS_CURRENT = "N", ' ||
```

```
    '    h.VALID_TO = TRUNC(SYSDATE), ' ||
```

```
    '    h.UPDATED_ON = SYSDATE ' ||
```

```
    'WHERE h.IS_CURRENT = "Y" ' ||
```

```
    'AND EXISTS ( ' ||
```



```

' SELECT 1 ' ||
' FROM OWN_AP12008.STG_MASTER_ROBI_LANDING I ' ||
' WHERE I.ROBID = h.ROBID ' ||
' AND ' || v_where_clause ||
');

EXECUTE IMMEDIATE v_sql;
v_expired := SQL%ROWCOUNT;
DBMS_OUTPUT.PUT_LINE('Step 1B - Expired in STG: ' || v_expired);

-- =====
-- STEP 1C: Handle departed ROBIDs (in STG but not in snapshot)
-- =====

INSERT INTO OWN_AP12008.STG_MASTER_ROBI_HIST
SELECT h.*
FROM OWN_AP12008.STG_MASTER_ROBI h
WHERE h.IS_CURRENT = 'Y'
AND NOT EXISTS (
    SELECT 1 FROM OWN_AP12008.STG_MASTER_ROBI_LANDING I
    WHERE I.ROBID = h.ROBID
);
DBMS_OUTPUT.PUT_LINE('Step 1C - Departed archived: ' || SQL%ROWCOUNT);

UPDATE OWN_AP12008.STG_MASTER_ROBI h
SET h.IS_CURRENT = 'N',
    h.VALID_TO = TRUNC(SYSDATE),
    h.UPDATED_ON = SYSDATE
WHERE h.IS_CURRENT = 'Y'
AND NOT EXISTS (
    SELECT 1 FROM OWN_AP12008.STG_MASTER_ROBI_LANDING I
    WHERE I.ROBID = h.ROBID
);
v_departed := SQL%ROWCOUNT;
DBMS_OUTPUT.PUT_LINE('Step 1C - Departed expired: ' || v_departed);

-- =====
-- STEP 2: MERGE — rebuild the WHERE clause with tgt/src aliases
-- for the MATCHED condition
-- =====

v_where_clause := OWN_AP12008.FN_BUILD_SCD2_WHERE_CLAUSE(
    v_table_name, v_source_name, 'tgt', 'src');

```

```

v_sql :=
'MERGE INTO OWN_AP12008.STG_MASTER_ROBI tgt ' ||
'USING ( ' ||
'  SELECT * FROM OWN_AP12008.STG_MASTER_ROBI_LANDING ' ||
') src ' ||
'ON (tgt.ROBID = src.ROBID AND tgt.IS_CURRENT = "Y") ' ||
'WHEN MATCHED THEN ' ||
'  UPDATE SET ' ||
'    tgt.FLID    = src.FLID, ' ||
'    tgt.START_DATE = src.START_DATE, ' ||
'    tgt.END_DATE  = src.END_DATE, ' ||
'    tgt.ENTITYID  = src.ENTITYID, ' ||
'    tgt.LASTNAME   = src.LASTNAME, ' ||
'    tgt.FIRSTNAME  = src.FIRSTNAME, ' ||
'    tgt.COUNTRY    = src.COUNTRY, ' ||
'    tgt.LANGUAGE   = src.LANGUAGE, ' ||
'    tgt.REFOG      = src.REFOG, ' ||
'    tgt.EMAIL      = src.EMAIL, ' ||
'    tgt.TYPE       = src.TYPE, ' ||
'    tgt.STATUS     = src.STATUS, ' ||
'    tgt.ISTEST     = src.ISTEST, ' ||
'    tgt.ISRPA      = src.ISRPA, ' ||
'    tgt.UPDATED_ON = SYSDATE ' ||
'  WHERE NOT ' || v_where_clause || ' ' ||
'WHEN NOT MATCHED THEN ' ||
'  INSERT ( ' ||
'    ROBID, FLID, START_DATE, END_DATE, ENTITYID, ' ||
'    ENTITY, MANAGER, LASTNAME, FIRSTNAME, COUNTRY, ' ||
'    LANGUAGE, FUNCTION, REFOG, EMAIL, TYPE, ' ||
'    STATUS, ISTEST, ISRPA, ' ||
'    SOURCE, CREATED_ON, UPDATED_ON, ' ||
'    VALID_FROM, VALID_TO, IS_CURRENT ' ||
'  ) VALUES ( ' ||
'    src.ROBID, src.FLID, src.START_DATE, src.END_DATE, src.ENTITYID, ' ||
'    src.ENTITY, src.MANAGER, src.LASTNAME, src.FIRSTNAME, src.COUNTRY, ' ||
'    src.LANGUAGE, src.FUNCTION, src.REFOG, src.EMAIL, src.TYPE, ' ||
'    src.STATUS, src.ISTEST, src.ISRPA, ' ||
'    "ROBI", SYSDATE, SYSDATE, ' ||
'    TRUNC(SYSDATE), DATE "9999-12-31", "Y" ' ||
'  );

```

```
EXECUTE IMMEDIATE v_sql;
```

```

v_merged := SQL%ROWCOUNT;
DBMS_OUTPUT.PUT_LINE('Step 2 - Rows merged into STG: ' || v_merged);

COMMIT;

DBMS_OUTPUT.PUT_LINE(v_proc_name || ' completed at: ' ||
    TO_CHAR(SYSTIMESTAMP, 'YYYY-MM-DD HH24:MI:SS'));

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(v_proc_name || ' FAILED: ' || SQLERRM);
        DBMS_OUTPUT.PUT_LINE('Failed SQL: ' || v_sql);
        RAISE;

END SP_LOAD_STG_MASTER_ROBI;
/

```

How to add a new tracked column in future

->Zero code changes — just one SQL:

```

INSERT INTO OWN_AP12008.CFG_SCD2_TRACKED_COLUMNS
    (TABLE_NAME, SOURCE_NAME, COLUMN_NAME, COMMENTS)
VALUES ('STG_MASTER_ROBI', 'ROBI', 'COUNTRY', 'Track country changes');

COMMIT;

```

Next run picks it up automatically.

To disable a tracked column temporarily:

```

UPDATE OWN_AP12008.CFG_SCD2_TRACKED_COLUMNS
SET   IS_ACTIVE = 'N'
WHERE TABLE_NAME = 'STG_MASTER_ROBI'
AND   SOURCE_NAME = 'ROBI'
AND   COLUMN_NAME = 'MANAGER';
COMMIT;

```